

3. AI Large Model Voice Interaction

3. AI Large Model Voice Interaction

- 1. Concept Introduction
 - 1.1 What is "AI Large Model Voice Interaction"?
 - 1.2 Brief Description of Implementation Principles
- 2. Project Architecture
 - 2.1 Key Code Analysis
- 3. Practical Operations
 - 3.1 Configuring Online LLM
 - 3.2 Startup and Testing

1. Concept Introduction

1.1 What is "AI Large Model Voice Interaction"?

In the `largemodel` project, **AI Large Model Voice Interaction** connects the previously introduced **Offline ASR** and **Offline TTS** with the **Large Language Model (LLM)** core, forming a complete conversational system that can listen, speak, and think.

This is no longer an isolated function, but the prototype of a true **voice assistant**. Users can have natural language conversations with the robot through voice, and the robot can understand questions, think about answers, and respond with voice. The entire process is completed locally without requiring network access.

The core of this function is the `model_service` **ROS2 node**. It acts as the brain and neural center, subscribing to ASR recognition results, invoking the LLM for thinking, and then publishing the LLM's text replies to the TTS node for speech synthesis.

1.2 Brief Description of Implementation Principles

The implementation principle of this function is a classic data pipeline:

1. **Voice -> Text (ASR)**: The `asr` node continuously listens to ambient sounds. Once it detects that the user has finished speaking, it converts the audio to text and publishes it to the `/asr_text` topic.
2. **Text -> Thinking -> Text (LLM)**: The `model_service` node subscribes to the `/asr_text` topic. When it receives text from ASR, it sends it as a prompt to the locally deployed large language model (such as `qwen` running through `ollama`). The LLM generates a response text based on the context.
3. **Text -> Voice (TTS)**: After the `model_service` node gets the LLM's response, it publishes it to the `/tts_text` topic.
4. **Voice Playback**: The `tts_only` node subscribes to the `/tts_text` topic. Upon receiving the text, it immediately invokes the offline TTS model to synthesize it into audio and plays it through the speaker.

This process forms a complete closed loop of **Voice Input -> Text Processing -> Text Output -> Voice Output**.

2. Project Architecture

2.1 Key Code Analysis

The core of the entire process lies in how the `model_service` node connects input and output.

1. Subscribe to ASR Results (located in `largemodel/model_service.py`)

The `model_service` node has a subscriber to receive text recognized by ASR.

```
# largemodel/model_service.py (Core logic illustration)
class ModelService(Node):
    def __init__(self):
        super().__init__('model_service')
        # ...
        # Subscribe to ASR text output topic
        self.asr_subscription = self.create_subscription(
            String,
            'asr_text',
            self.asr_callback,
            10)

        # Create TTS text input topic publisher
        self.tts_publisher = self.create_publisher(String, 'tts_text', 10)

        # Initialize large model interface
        self.large_model_interface = LargeModelInterface(self)
```

Explanation: The node's `__init__` method clearly defines its role: a middleman that can both "listen" to ASR results and "command" TTS to speak, with an internal "brain" (`LargeModelInterface`).

2. Process ASR Text and Call LLM (located in `largemodel/model_service.py`)

When ASR has new recognition results, `asr_callback` will be triggered.

```
# largemodel/model_service.py (Core logic illustration)
def asr_callback(self, msg):
    user_text = msg.data
    self.get_logger().info(f'Received from ASR: "{user_text}"')

    # Call large model interface for thinking
    # llm_platform determines whether to call Ollama or online API
    llm_platform = self.get_parameter('llm_platform').value
    response_text = self.large_model_interface.call_llm(user_text,
        llm_platform)

    if response_text:
        self.get_logger().info(f'LLM response: "{response_text}"')
        # Send LLM response to TTS
        self.speak(response_text)
```

Explanation: This is the core logic of the system. After the callback function receives the text, it immediately sends it to the LLM through `large_model_interface`. The `call_llm` method internally decides whether to connect to local Ollama or online API based on the `llm_platform` configuration.

3. Send LLM Response to TTS (located in `largemodel/model_service.py`)

The `speak` method is a simple wrapper for publishing text to the topic that the TTS node listens to.

```
# largemodel/model_service.py (Core logic illustration)
def speak(self, text):
    msg = String()
    msg.data = text
    self.tts_publisher.publish(msg)
```

Explanation: This function completes the final step of the data flow, passing the text result from the "brain" to the "mouth", thus completing the entire voice interaction closed loop.

3. Practical Operations

3.1 Configuring Online LLM

1. First obtain an API Key from any of the platforms in the previous tutorials
2. Then update the key in the configuration file, open the model interface configuration file `large_model_interface.yaml`:

```
vim
~/yahboom_ws/install/largemodel/share/largemodel/config/large_model_interfac
e.yaml
```

3. Enter your API Key:

Find the corresponding section and paste the API Key you just copied. Here we use Tongyi Qianwen configuration as an example.

```
# large_model_interface.yaml

## Tongyi Qianwen
qianwen_api_key: "sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx" # Paste your Key
qianwen_model: "qwen-v1-max-latest" # You can choose the model as needed,
e.g. qwen-turbo, qwen-plus
```

4. Open the main configuration file `yahboom.yaml`:

```
vim ~/yahboom_ws/install/largemodel/share/largemodel/config/yahboom.yaml
```

5. Select the online platform to use:

Change the `llm_platform` parameter to the platform name you want to use.

```
# yahboom.yaml

model_service:
  ros__parameters:
    # ...
    llm_platform: 'tongyi' #Optional platforms: 'ollama', 'tongyi',
'spark', 'qianfan', 'openrouter'
```

3.2 Startup and Testing

1. Start the `largemodel` main program:

Run the following command to start voice interaction:

```
ros2 launch largemodel largemodel_control.launch.py
```

2. Test:

- **Wake up:** Say to the microphone: "Hello, yahboom."
- **Conversation:** After the speaker responds, you can speak your question.
- **Observe logs:** In the terminal running the `launch` file, you should see:
 1. The ASR node recognizes your question and prints it out.
 2. The `model_service` node receives the text, invokes the LLM, and prints out the LLM's response.
- **Listen to the answer:** After a while, you should hear the answer from the speaker.

Possible issue: Hello yahboom can wake up but audio cannot be captured

Solution:

- Confirm USB microphone is recognized by the system

```
arecord -l
```

Check if "USB Audio Device" appears in the output

- Confirm `mic_index` configuration

```
grep mic_index  
~/yahboom_ws/install/largemodel/share/largemodel/config/yahboom.yaml
```

Recommended to use 0 (uses PulseAudio default source, automatic resampling)

- Confirm PulseAudio default source is USB microphone

```
pactl get-default-source
```

If not, first check the source name:

```
pactl list short sources | grep usb
```

Then set the default source:

```
pactl set-default-source <source_name>
```

- Start testing

```
ros2 launch largemodel largemodel_control.launch.py
```

